

```
>>> id(spam), spam
(38329712, 'Hello world!')
>>> spam = spam.upper()
>>> id(spam), spam
(38329648, 'HELLO WORLD!')
>>> spam = 'Hi'
>>> id(spam), spam
(38395568, 'Hi')
>>> spam = f'{spam} world!'
>>> id(spam), spam
(38330864, 'Hi world!')
```

Notice that each call to `id(spam)` returns a different identity, because the string object in `spam` isn't being changed: it's being replaced by a whole new string object with a different identity. Creating new strings by using f-strings, the `format()` string method, or the `%s` format specifiers also creates new string objects, just like string concatenation. Normally, this technical detail doesn't matter. Python is a high-level language that handles many of these details for you so you can focus on creating your program.

But building a string through a large number of string concatenations can slow down your programs. Each iteration of the loop creates a new string object and discards the old string object: in code, this looks like concatenations inside a `for` or `while` loop, as in the following:

```
>>> finalString = ''
>>> for i in range(100000):
...     finalString += 'spam '
...
>>> finalString
spam spam spam spam spam spam spam spam spam spam spam spam --snip--
```

Because the `finalString += 'spam '` happens 100,000 times inside the loop, Python is performing 100,000 string concatenations. The CPU has to create these intermediate string values by concatenating the current `finalString` with `'spam '`, put them into memory, and then almost immediately discard them on the next iteration. This is a lot of wasted effort, because we only care about the final string.

The pythonic way to build strings is to append the smaller strings to a list and then join the list together into one string. This method still creates